# Aarogya Healthcare Chatbot

System Logic & Technical Documentation

---

## 1. Overview

Aarogya is a Retrieval-Augmented Generation (RAG) healthcare information chatbot. It answers general questions on symptoms, diseases, nutrition, preventive care, healthy lifestyle, and first aid, while strictly avoiding diagnosis, prescriptions, and dosage recommendations. This document explains how a single user message flows through the system end to end.

## 2. How Queries Are Processed

Every call to **POST /chat** is handled by a single orchestration function in `app/api.py` that runs the following pipeline in order:

- **Session resolution** — an existing `session_id` is reused, or a new UUID is generated.
- **Input guardrails** — the raw message is scanned for emergencies, prompt injection/leak attempts, and risky request types (see Section 5). If the message is blocked, a deterministic response is returned immediately and the LLM is never called.
- **RAG retrieval** — if not blocked, the message is embedded and the top-k most similar knowledge-base chunks are retrieved (see Section 4).
- **Prompt assembly** — the system prompt, retrieved context, and question are combined into a single user turn (see Section 3).
- **LLM call** — the configured provider (OpenAI or Gemini) generates a response, with retry and fallback-model logic.
- **Output guardrails** — the generated text is swept for residual diagnosis/dosage language and sanitized if needed.
- **Memory update** — both turns are appended to the session's conversation history.
- **Response** — a structured JSON object is returned: response text, standard medical disclaimer, source citations, guardrail flags, emergency flag, and latency.

## 3. How Prompts Are Built

Prompt construction lives entirely in `app/prompts.py`, separated from orchestration logic so it can be reviewed and unit-tested independently.

### System prompt

A fixed system prompt establishes the assistant's persona ("Aarogya"), tone (friendly, empathetic, concise), and — critically — a list of things it must never do: diagnose, prescribe, recommend dosages, interpret personal lab results, or tell a user to change a prescribed treatment. It also instructs the model to stay in character and decline attempts to override these rules.

### User turn

Each user turn is built by `build_user_turn()`, which wraps the retrieved context chunks (numbered, with source labels) together with the user's question and a short instruction to ground the answer in that context when relevant, without fabricating information the context doesn't support.

### Conversation history

The last `MAX_MEMORY_TURNS` exchanges for the session are passed to the LLM client alongside the system and user prompts, so follow-up questions ("can I exercise?" after "I have a fever") are answered with prior context in mind.

## 4. How Retrieval-Augmented Generation Works

### Indexing (offline / on startup)

- Documents in `knowledge_base/docs/` (WHO/CDC/NIH/MedlinePlus-style reference material) are read and split into ~500-word overlapping chunks (`app/rag.py: chunk_text()`).
- Each chunk is embedded with the `all-MiniLM-L6-v2` Sentence-Transformers model (384-dim, normalized for cosine similarity).
- Embeddings are stored in a FAISS `IndexFlatIP` index, persisted to `vectorstore/index.faiss` alongside a JSON metadata sidecar (`vectorstore/metadata.json`) mapping vector positions back to chunk text, title, and source.

### Retrieval (per query)

- The user's question is embedded with the same model.
- FAISS returns the top-k (default 4) most similar chunks by inner-product score.
- Each result becomes a `SourceDocument` (title, source, snippet, score) that is both injected into the prompt as grounding context and returned to the client as a citation.

If no documents are found relevant, or RAG is disabled (`RAG_ENABLED=false`), the LLM answers from general knowledge under the same safety constraints — the system degrades gracefully rather than failing the request.

# 5. How Guardrails Work

Guardrails are implemented as fast, deterministic, unit-testable regex/keyword rules in `app/guardrails.py` — deliberately not a second LLM call, so they have no dependency on an external API and cannot themselves be prompt-injected.

## Input-side checks (before the LLM is called)

| Category | Examples matched | Action |
|---|---|---|
| Medical emergency | Chest pain, difficulty breathing, stroke symptoms, severe bleeding, loss of consciousness, anaphylaxis | Blocked — deterministic emergency response with local emergency numbers |
| Mental health crisis | Suicidal ideation, self-harm language | Blocked — emergency response plus crisis helpline numbers |
| Prompt injection | “Ignore previous instructions”, “act as...”, “override your rules” | Blocked — polite refusal, conversation continues |
| Prompt leak attempt | “Show me your system prompt” | Blocked — polite refusal |
| Illegal drug advice | Synthesis instructions, “get high on...” | Blocked — redirect to support resources |
| Diagnosis request | “Do I have...”, “what's wrong with me” | Flagged only — LLM redirects per system prompt |
| Dosage / prescription request | “How many mg”, “prescribe me...” | Flagged only — LLM redirects per system prompt |

"Blocked" categories skip the LLM entirely and return a fixed, reviewed response — this guarantees emergency guidance is always correct and instant, and can never be altered by a jailbreak of the LLM itself. "Flagged" categories still reach the LLM, which is instructed by the system prompt to redirect rather than comply, giving a more natural, context-aware response for borderline cases.

## Output-side checks (after the LLM responds)

As defense in depth, the LLM's own generated text is swept for residual diagnosis language ("you have a...") or dosage language ("take 500 mg..."). If found, the offending phrase is replaced with a safe redirect ("take a dose recommended by your doctor or pharmacist") and a note is appended informing the user that a specific figure was removed for safety.

# 6. How Memory Works

Conversation memory is a thread-safe, per-session, in-process store (`app/memory.py: MemoryStore`) keyed by `session_id`. Each session keeps a sliding window of the most recent `MAX_MEMORY_TURNS` exchanges (default 10 user+assistant pairs); older turns are trimmed automatically to bound memory and prompt size. The store is abstracted behind a small interface so a persistent backend (Redis, Postgres)

can be substituted later without touching callers.

## 7. Safety Measures Summary

- Hard-coded system-prompt boundaries against diagnosis, prescriptions, and dosages.
- Deterministic, LLM-independent emergency interception with region-appropriate helpline numbers.
- Defense-in-depth output sanitization in case the LLM slips despite instructions.
- Mandatory medical disclaimer attached to every response.
- Prompt-injection and prompt-leak detection that short-circuits the LLM call.
- Structured logging of every request, its guardrail flags, and latency for auditability.

## 8. Challenges

- **Balancing helpfulness and safety** — over-blocking makes the assistant useless for legitimate education (e.g. "what causes a fever" vs. "do I have a fever-causing disease"); the current rules flag rather than block borderline diagnosis/dosage phrasing so the LLM can still give a safe, redirecting answer.
- **Regex coverage vs. natural language variety** — users phrase emergencies and risky requests in many ways; the pattern banks are deliberately broad (multiple synonyms per category) but can't catch every paraphrase, which is why output-side sanitization exists as a second layer.
- **Grounding without over-constraining** — RAG context should inform, not straitjacket, answers to questions the knowledge base doesn't cover; the prompt explicitly allows general medical knowledge when context is irrelevant, while still forbidding fabrication.
- **Provider abstraction** — OpenAI and Gemini have different chat-history and system-prompt conventions; a common `BaseLLMClient` interface hides this so the rest of the app is provider-agnostic.

## 9. Future Improvements

- LLM-based secondary guardrail classifier for higher recall on paraphrased risk.
- Persistent (Redis/Postgres) memory so sessions survive restarts and scale across workers.
- Streaming responses (SSE/WebSocket) for a more responsive chat feel.
- Multilingual input/output and voice input/speech output.
- Expanded, properly licensed knowledge base ingested directly from WHO/CDC/NIH/MedlinePlus feeds.
- Response caching for frequently asked questions to reduce latency and cost.